

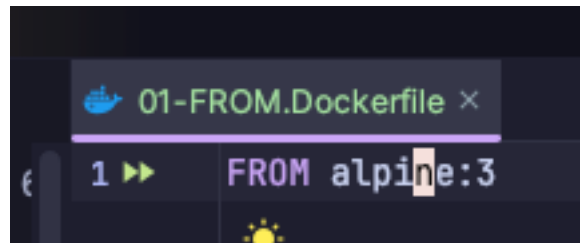
Tugas Praktikum 2 - Dockerfile

11231015 - Bagus Nur Andiansyah

2026-05-19

Link source code:

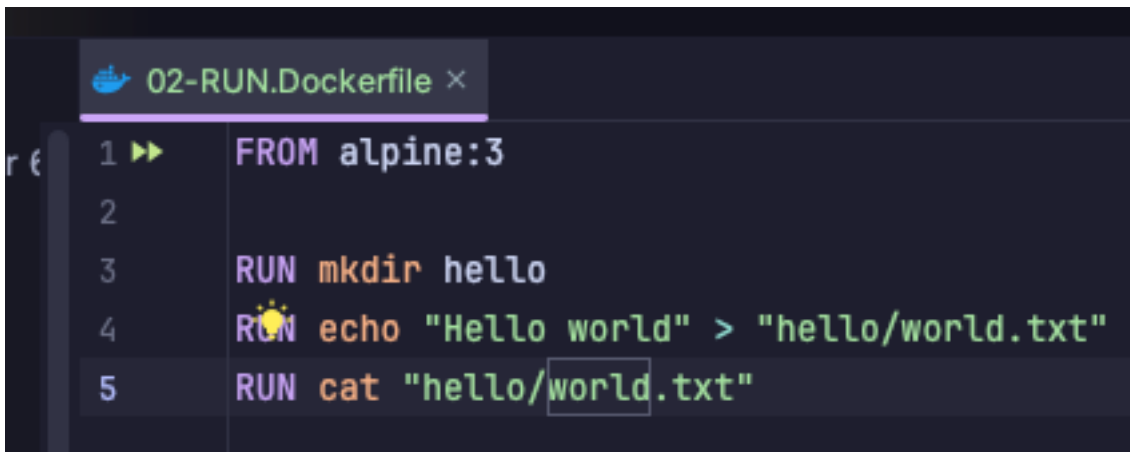
FROM Instruction



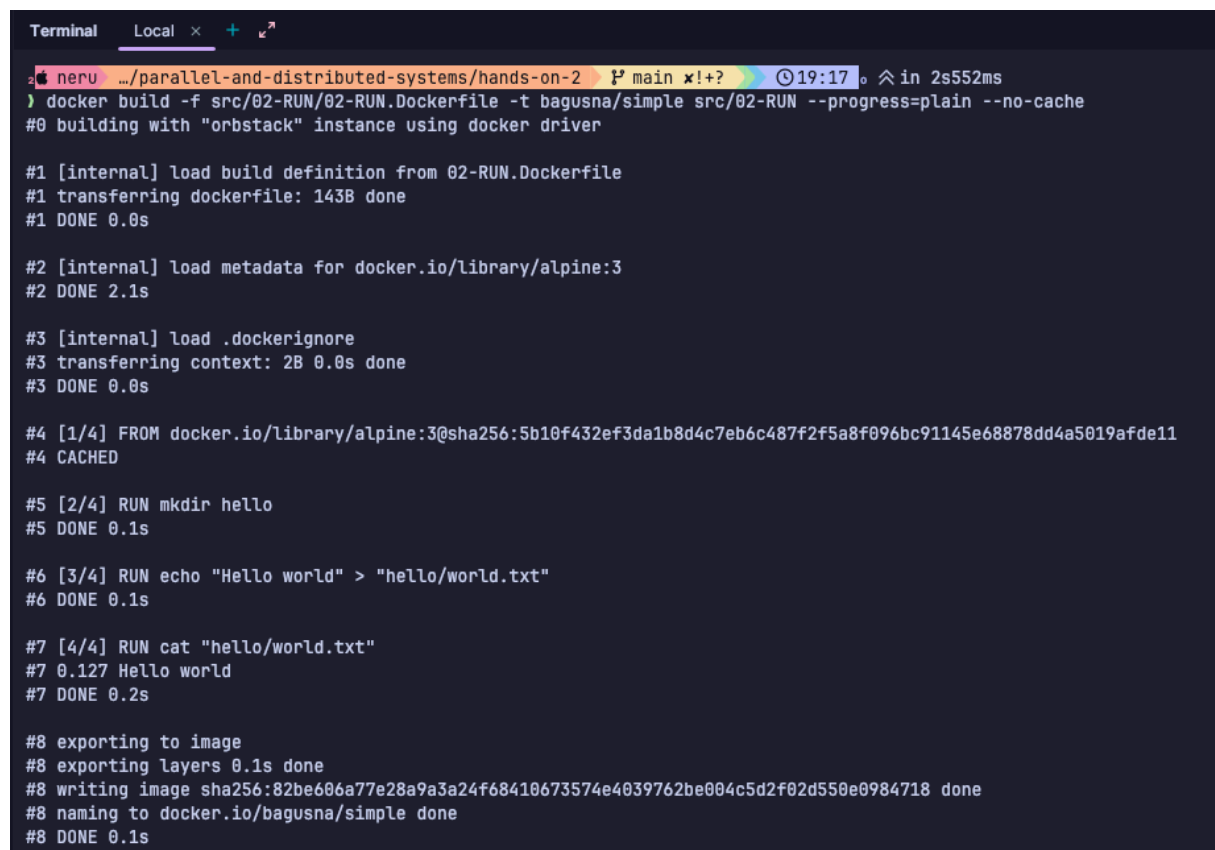
```
Terminal Local x + ↗
neru .../parallel-and-distributed-systems/hands-on-2 main x!+? 19:15
) docker build -f src/01-FROM/01-FROM.Dockerfile -t bagusna/simple src/01-FROM
[+] Building 5.1s (5/5) FINISHED
=> [internal] load build definition from 01-FROM.Dockerfile
=> => transferring dockerfile: 58B
=> [internal] load metadata for docker.io/library/alpine:3
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/1] FROM docker.io/library/alpine:3@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
=> => resolve docker.io/library/alpine:3@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
=> => sha256:378c4c5418f7493bd500ad21ffb43818d0689daaad43e3261859fb417d1481a0 1.02kB / 1.02kB
=> => sha256:2fffb2ff4aab36d06b7f3266bbb10e8232769cd2360613131d37abd19430cf6f1 627B / 627B
=> => sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11 9.22kB / 9.22kB
=> exporting to image
=> => exporting layers
=> => writing image sha256:a3f03f25a571ff2bc2c4f3f34bbdfd73b3306c8006278a67d39470ac7f0269d6
=> => naming to docker.io/bagusna/simple
```

FROM merupakan instruksi yang umum digunakan pertama kali dalam membangun Docker image. Docker image yang dibangun pada umumnya menggunakan image lain sebagai *base*-nya. Command `docker build` digunakan untuk mem-*build* Dockerfile yang telah dibuat.

RUN Instruction



```
02-RUN.Dockerfile x
1 FROM alpine:3
2
3 RUN mkdir hello
4 RUN echo "Hello world" > "hello/world.txt"
5 RUN cat "hello/world.txt"
```



```
neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:17 in 2s552ms
) docker build -f src/02-RUN/02-RUN.Dockerfile -t bagusna/simple src/02-RUN --progress=plain --no-cache
#0 building with "orbstack" instance using docker driver

#1 [internal] load build definition from 02-RUN.Dockerfile
#1 transferring dockerfile: 143B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/alpine:3
#2 DONE 2.1s

#3 [internal] load .dockerignore
#3 transferring context: 2B 0.0s done
#3 DONE 0.0s

#4 [1/4] FROM docker.io/library/alpine:3@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
#4 CACHED

#5 [2/4] RUN mkdir hello
#5 DONE 0.1s

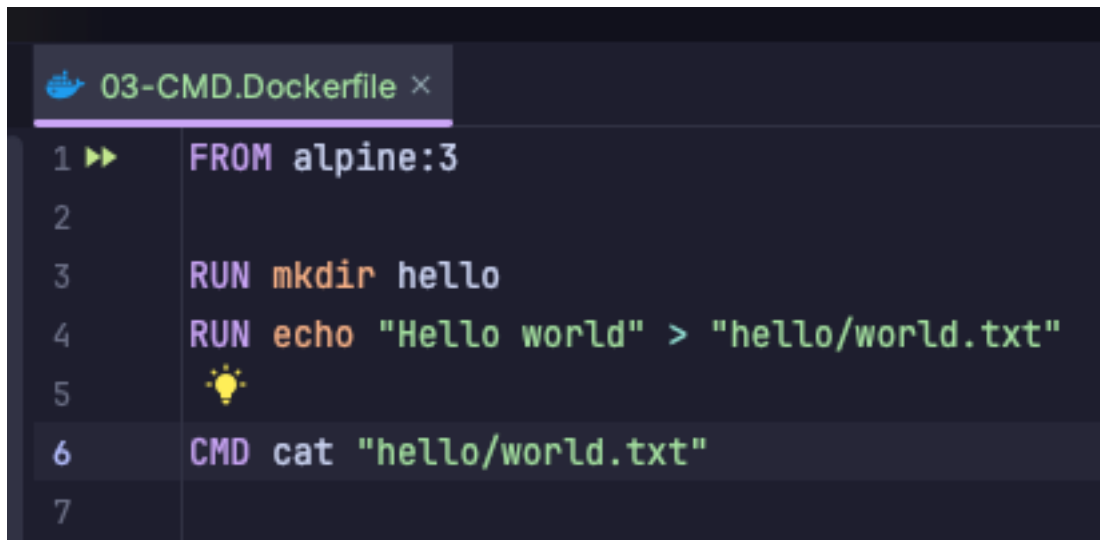
#6 [3/4] RUN echo "Hello world" > "hello/world.txt"
#6 DONE 0.1s

#7 [4/4] RUN cat "hello/world.txt"
#7 0.127 Hello world
#7 DONE 0.2s

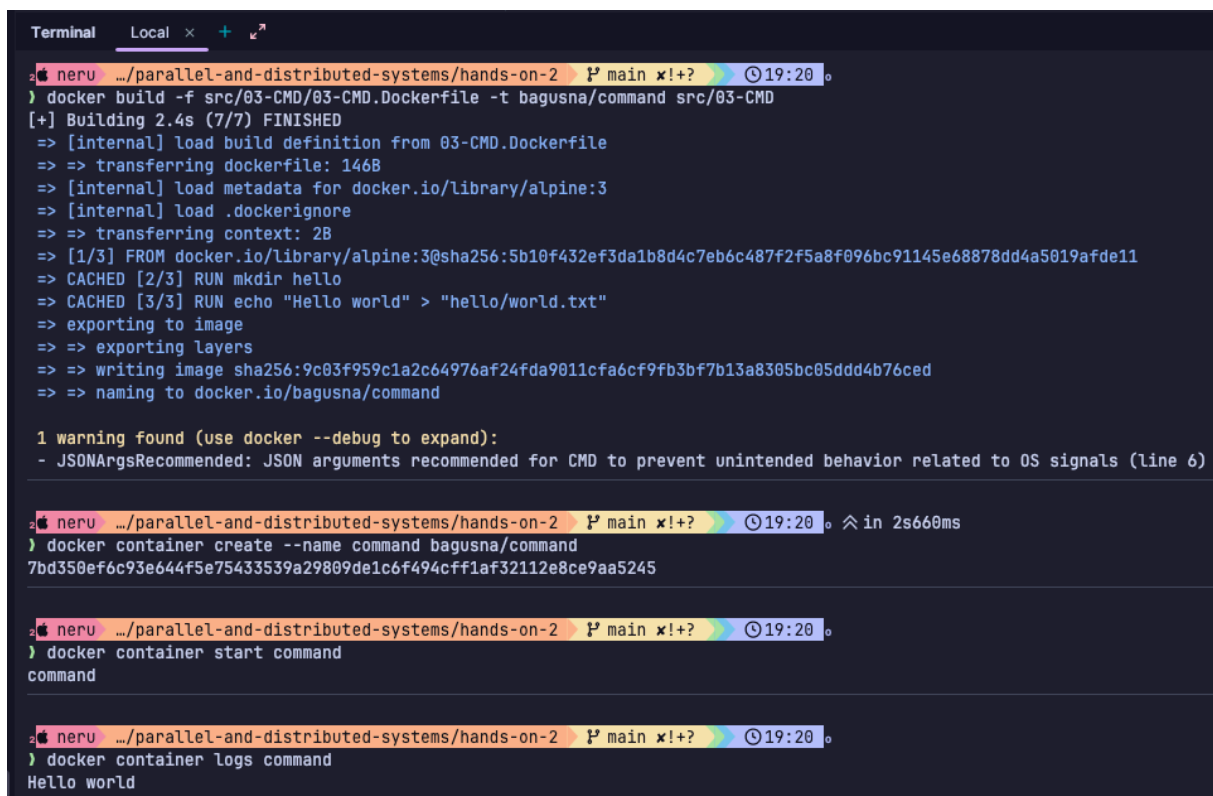
#8 exporting to image
#8 exporting layers 0.1s done
#8 writing image sha256:82be606a77e28a9a3a24f68410673574e4039762be004c5d2f02d550e0984718 done
#8 naming to docker.io/bagusna/simple done
#8 DONE 0.1s
```

Instruksi RUN digunakan untuk menjalankan command ketika proses pembuatan image berjalan. Dalam contoh ini dijalankan beberapa command yang outputnya dapat dilihat selama proses *image building*.

CMD Instruction



```
03-CMD.Dockerfile x
1 FROM alpine:3
2
3 RUN mkdir hello
4 RUN echo "Hello world" > "hello/world.txt"
5
6 CMD cat "hello/world.txt"
7
```



```
Terminal Local x +
neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:20
) docker build -f src/03-CMD/03-CMD.Dockerfile -t bagusna/command src/03-CMD
[+] Building 2.4s (7/7) FINISHED
=> [internal] load build definition from 03-CMD.Dockerfile
=> == transferring dockerfile: 146B
=> [internal] load metadata for docker.io/library/alpine:3
=> [internal] load .dockerignore
=> == transferring context: 2B
=> [1/3] FROM docker.io/library/alpine:3@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
=> CACHED [2/3] RUN mkdir hello
=> CACHED [3/3] RUN echo "Hello world" > "hello/world.txt"
=> exporting to image
=> == exporting layers
=> == writing image sha256:9c03f959c1a2c64976af24fda9011cfa6cf9fb3bf7b13a8305bc05ddd4b76ced
=> == naming to docker.io/bagusna/command

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 6)

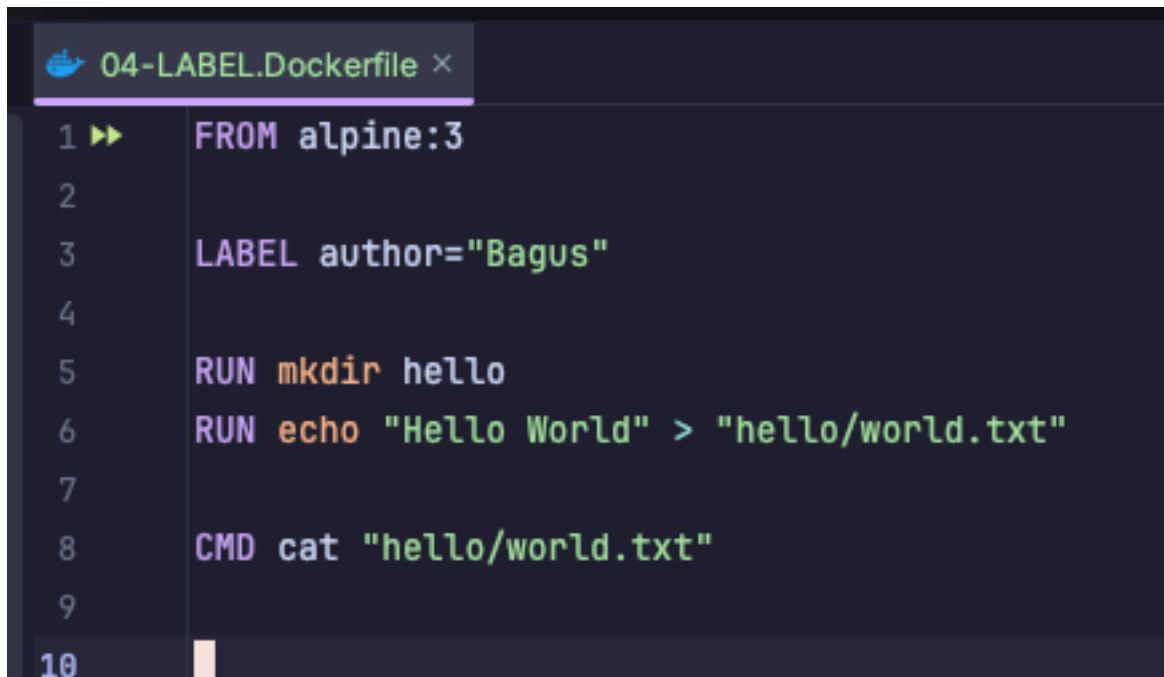
neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:20 in 2s660ms
) docker container create --name command bagusna/command
7bd350ef6c93e644f5e75433539a29809de1c6f494cff1af32112e8ce9aa5245

neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:20
) docker container start command
command

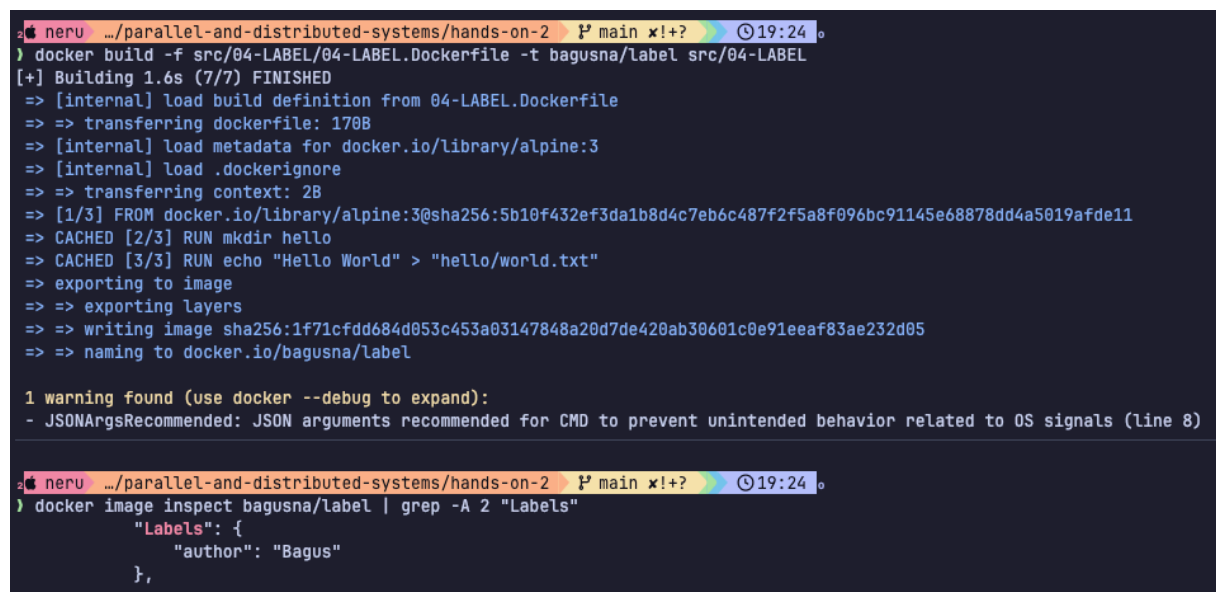
neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:20
) docker container logs command
Hello world
```

CMD merupakan instruksi yang berisi command yang akan berjalan ketika container dijalankan, bukan dalam proses *image building*. Dalam contoh ini, terlihat command `cat` berjalan hanya setelah container dijalankan.

LABEL Instruction



```
04-LABEL.Dockerfile x
1  FROM alpine:3
2
3  LABEL author="Bagus"
4
5  RUN mkdir hello
6  RUN echo "Hello World" > "hello/world.txt"
7
8  CMD cat "hello/world.txt"
9
10
```



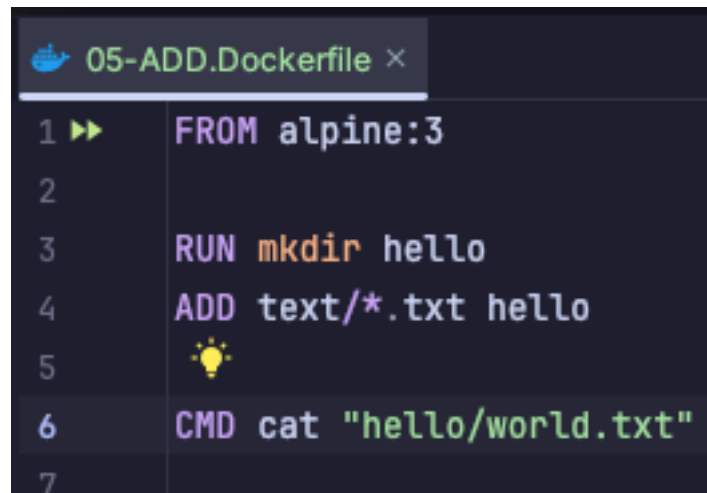
```
neru ~/parallel-and-distributed-systems/hands-on-2 1' main x!+? 19:24
$ docker build -f src/04-LABEL/04-LABEL.Dockerfile -t bagusna/label src/04-LABEL
[+] Building 1.6s (7/7) FINISHED
=> [internal] load build definition from 04-LABEL.Dockerfile
=> => transferring dockerfile: 170B
=> [internal] load metadata for docker.io/library/alpine:3
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/alpine:3@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
=> CACHED [2/3] RUN mkdir hello
=> CACHED [3/3] RUN echo "Hello World" > "hello/world.txt"
=> exporting to image
=> => exporting layers
=> => writing image sha256:1f71cfdd684d053c453a03147848a20d7de420ab30601c0e91eeaf83ae232d05
=> => naming to docker.io/bagusna/label

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 8)

neru ~/parallel-and-distributed-systems/hands-on-2 1' main x!+? 19:24
$ docker image inspect bagusna/label | grep -A 2 "Labels"
  "Labels": {
    "author": "Bagus"
  },
```

LABEL digunakan untuk menyematkan metadata ke dalam image yang dibuat. Metadata dapat berisi apapun, misalnya author, lisensi, dan tautan website.

ADD Instruction



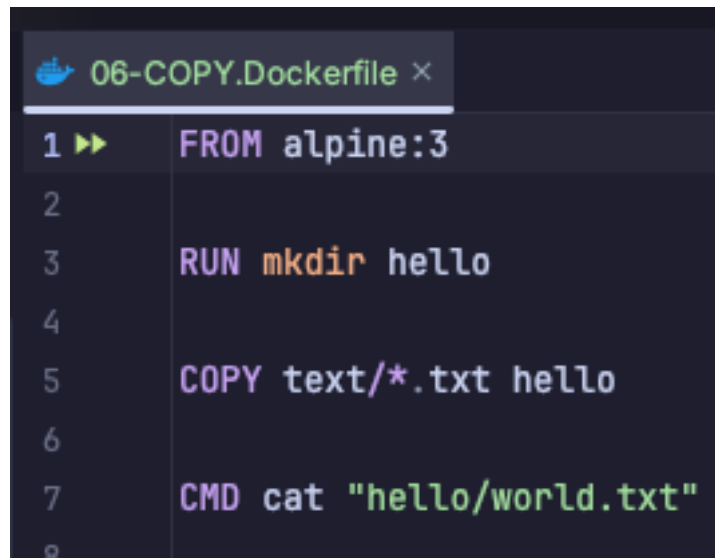
```
05-ADD.Dockerfile x
1  FROM alpine:3
2
3  RUN mkdir hello
4  ADD text/*.txt hello
5  
6  CMD cat "hello/world.txt"
7
```



```
neru .../parallel-and-distributed-systems/hands-on-2 > main x!+? 19:26 in 2s643ms
) docker build -f src/05-ADD/05-ADD.Dockerfile -t bagusna/add src/05-ADD
[+] Building 1.1s (8/8) FINISHED
=> [internal] load build definition from 05-ADD.Dockerfile
=> => transferring dockerfile: 124B
=> [internal] load metadata for docker.io/library/alpine:3
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/alpine:3@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
=> [internal] load build context
=> => transferring context: 74B
=> CACHED [2/3] RUN mkdir hello
=> [3/3] ADD text/*.txt hello
=> exporting to image
=> => exporting layers
=> => writing image sha256:b90d39fb2f7e64b3f4a8a16f12a10f077a9e1de12baafb31bebc71322daa4620
=> => naming to docker.io/bagusna/add
```

ADD digunakan untuk menambahkan file ke dalam *image* yang dibangun. ADD dapat menerima file maupun url, serta mampu meng-*extract file archive*.

COPY Instruction



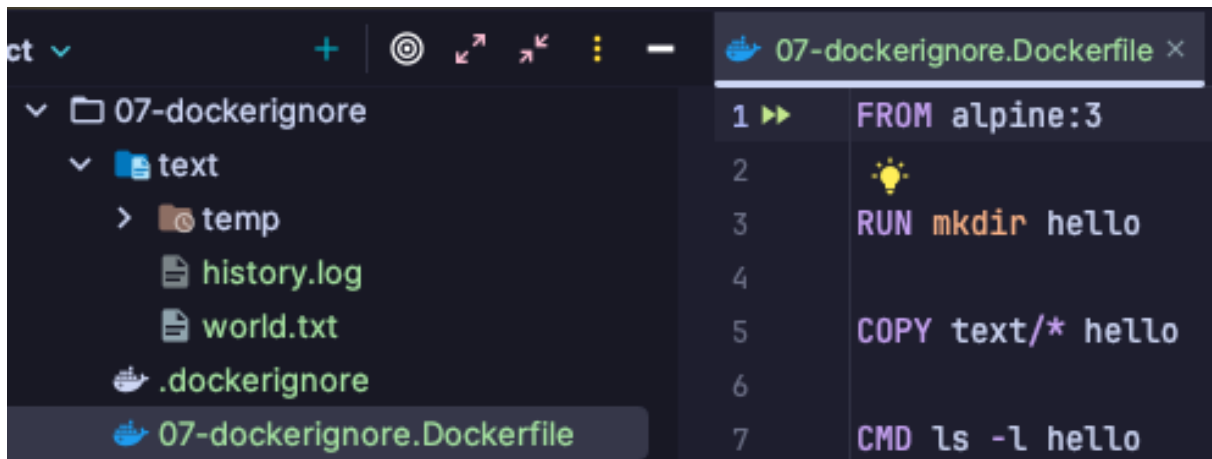
```
06-COPY.Dockerfile x
1  FROM alpine:3
2
3  RUN mkdir hello
4
5  COPY text/*.txt hello
6
7  CMD cat "hello/world.txt"
8
```



```
neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:26
> docker build -f src/06-COPY/06-COPY.Dockerfile -t bagusna/copy src/06-COPY
[+] Building 2.1s (8/8) FINISHED
=> [internal] load build definition from 06-COPY.Dockerfile
=> => transferring dockerfile: 126B
=> [internal] load metadata for docker.io/library/alpine:3
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/alpine:3@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
=> [internal] load build context
=> => transferring context: 74B
=> CACHED [2/3] RUN mkdir hello
=> [3/3] COPY text/*.txt hello
=> exporting to image
=> => exporting layers
=> => writing image sha256:2b2872384520b377193366f370edd627c1582c9fbe0674492d8c0e74c5f498a8
=> => naming to docker.io/bagusna/copy
```

Mirip dengan ADD, COPY dapat menambahkan file dari *host machine* ke dalam *image* yang dibangun. Perbedaananya COPY hanya menerima file saja. Instruction ini sering digunakan untuk menambahkan *source code* atau dependensi lain ke dalam *image*.

.dockerignore



File `.dockerignore` berisikan daftar file dan folder yang akan dikecualikan dari command COPY dan ADD. Dalam contoh ini, folder `temp` dan semua `*.log` dikecualikan dalam `.gitignore` sehingga tidak muncul dalam hasil image yang dibuat.

EXPOSE Instruction

```

1  FROM golang:1.18-alpine
2
3  RUN mkdir app
4  COPY main.go app
5
6  EXPOSE 8080
7
8  CMD go run app/main.go

```

```

neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:41
) docker build -f src/08-EXPOSE.Dockerfile -t bagusna/expose src/08-EXPOSE
[+] Building 190.2s (8/8) FINISHED
=> [internal] load build definition from 08-EXPOSE.Dockerfile
=> => transferring dockerfile: 139B
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> => resolve docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> => sha256:a9aaa45ef418e883481a13c7d84fa9904f2ec56789c52a87ba5a9e6483f2b74f 3.26MB / 3.26MB
=> => sha256:da3aa66ae7ca151fbd33126b15fa4d01bd5ceb35d53de36a8c6c82ecde58b596 286.26kB / 286.26kB
=> => sha256:72d1fe56e2997391a3dee0aa70cbe76557be6201cca145ad63b40e3767e061 110.45MB / 110.45MB
=> => sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807 1.65kB / 1.65kB
=> => sha256:670182396fe94e1b50ea107b66d4b2d158b2cd7c7bf93e1a53064a5b208c9930 1.16kB / 1.16kB
=> => sha256:6867eb9d8bd48c0f43214b09486a3dbf916437bd6ae140482ecb0301b3f24071 5.03kB / 5.03kB
=> => sha256:a13fd264aa8ef04a29b827fe2454ac2bb9c52f70bf0b0eadfb45ca4867190c00 156B / 156B
=> => extracting sha256:a9aaa45ef418e883481a13c7d84fa9904f2ec56789c52a87ba5a9e6483f2b74f
=> => extracting sha256:da3aa66ae7ca151fbd33126b15fa4d01bd5ceb35d53de36a8c6c82ecde58b596
=> => extracting sha256:72d1fe56e2997391a3dee0aa70cbe76557be6201cca145ad63b40e3767e061
=> => extracting sha256:a13fd264aa8ef04a29b827fe2454ac2bb9c52f70bf0b0eadfb45ca4867190c00
=> [internal] load build context
=> => transferring context: 277B
=> [2/3] RUN mkdir app
=> [3/3] COPY main.go app
=> exporting to image
=> => exporting layers
=> => writing image sha256:21532ae0849fc50cda9427e5bdf8cb12fc1be4b5427a770ce406fd1eeb55e985
=> => naming to docker.io/bagusna/expose

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 8)

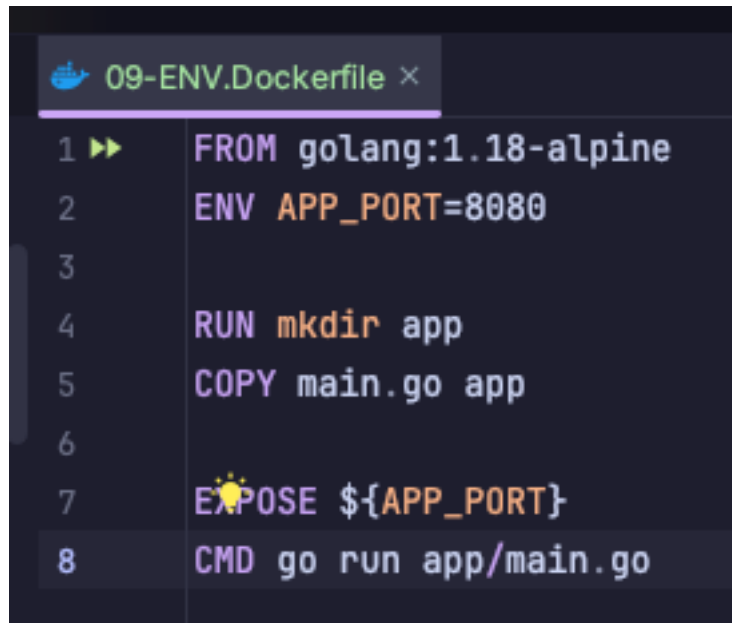
neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:45 in 3m10s581ms
) docker image inspect bagusna/expose | grep -B 2 "ExposedPorts"
    "Created": "2026-05-19T19:45:19.722453084+08:00",
    "Config": {
      "ExposedPorts": {

neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 19:46
) docker image inspect bagusna/expose | grep -A 2 "ExposedPorts"
      "ExposedPorts": {
        "8080/tcp": {}
      },

```


Instruksi EXPOSE digunakan untuk mendokumentasikan port dan protocol mana yang dapat di-*listen* dari container. Dalam contoh ini digunakan port 8080 untuk di-*expose*, terlihat dalam hasil inspect image.

ENV Instruction



```

1 FROM golang:1.18-alpine
2 ENV APP_PORT=8080
3
4 RUN mkdir app
5 COPY main.go app
6
7 EXPOSE ${APP_PORT}
8 CMD go run app/main.go

```



```

neru ~/parallel-and-distributed-systems/hands-on-2 > main x!+? 19:54
) docker build -f src/09-ENV/09-ENV.Dockerfile -t bagusna/env src/09-ENV
[+] Building 2.5s (8/8) FINISHED
=> [internal] load build definition from 09-ENV.Dockerfile
=> => transferring dockerfile: 160B
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> [internal] load build context
=> => transferring context: 29B
=> CACHED [2/3] RUN mkdir app
=> CACHED [3/3] COPY main.go app
=> exporting to image
=> => exporting layers
=> => writing image sha256:19ab0ff198e9aa4ee2f44d577d0db97789bb1945caf80a2a17dfac72791ba7b1
=> => naming to docker.io/bagusna/env

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 8)

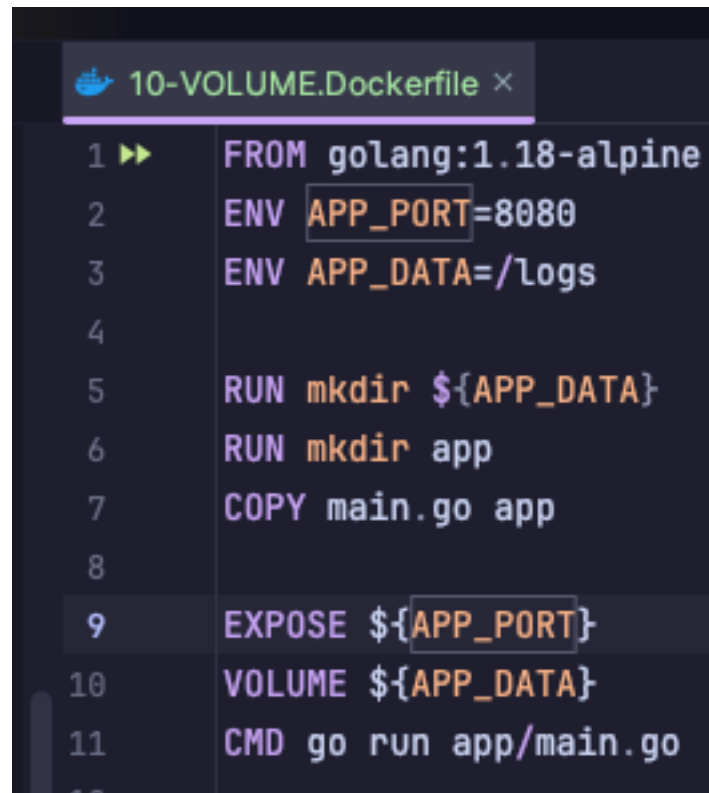
neru ~/parallel-and-distributed-systems/hands-on-2 > main x!+? 19:54
) docker image inspect bagusna/env | grep -A 5 "Env"
"Env": [
  "PATH=/go/bin:/usr/local/go/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
  "GOLANG_VERSION=1.18.10",
  "GOPATH=/go",
  "APP_PORT=8080"
],

neru ~/parallel-and-distributed-systems/hands-on-2 > main x!+? 19:54
) docker container create --name env --env APP_PORT=8080 -p 8080:8080 bagusna/env && docker container start env && docker container logs env
999537a1b2b6e33b57c62ea448245896bd0197ca1a2d02502caa70689c99c1a4
env

```

ENV digunakan untuk mendeklarasikan *environment variable* yang dapat digunakan dalam proses *building image*. Untuk menggunakan ENV yang telah dibuat, dapat menggunakan syntax `${NAMA_ENV}` ENV dapat ditimpa ketika membuat container dengan dengan option `--env key=value`.

VOLUME Instruction



```

1  FROM golang:1.18-alpine
2  ENV APP_PORT=8080
3  ENV APP_DATA=/logs
4
5  RUN mkdir ${APP_DATA}
6  RUN mkdir app
7  COPY main.go app
8
9  EXPOSE ${APP_PORT}
10 VOLUME ${APP_DATA}
11 CMD go run app/main.go

```

```

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 19:59
) docker build -f src/10-VOLUME/10-VOLUME.Dockerfile -t bagusna/volume src/10-VOLUME
[+] Building 3.4s (9/9) FINISHED
=> [internal] load build definition from 10-VOLUME.Dockerfile
=> => transferring dockerfile: 226B
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/4] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> [internal] load build context
=> => transferring context: 657B
=> [2/4] RUN mkdir /logs
=> [3/4] RUN mkdir app
=> [4/4] COPY main.go app
=> exporting to image
=> => exporting layers
=> => writing image sha256:a34254bf87bbbd4eaba4b58c62325e10e0ddb814b00a3eea7b470755470a735e
=> => naming to docker.io/bagusna/volume

```

```

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 11)

```

```

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 19:59
) docker image inspect bagusna/volume | grep -A 2 "Volumes"
    "Volumes": {
        "/logs": {}
    },

```

```

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:00
) docker container create --name volume --env APP_PORT=8080 -p 8080:8080 bagusna/volume
c103c600a961c891c4a43961952147de6d2ea50c9e33098e81da96e4c742220a

```

```

neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 20:00
> docker container start volume && docker container logs volume
volume

neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 20:00
> docker container logs volume
Run app in port : 8080

neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 20:00
> docker container logs volume
Run app in port : 8080

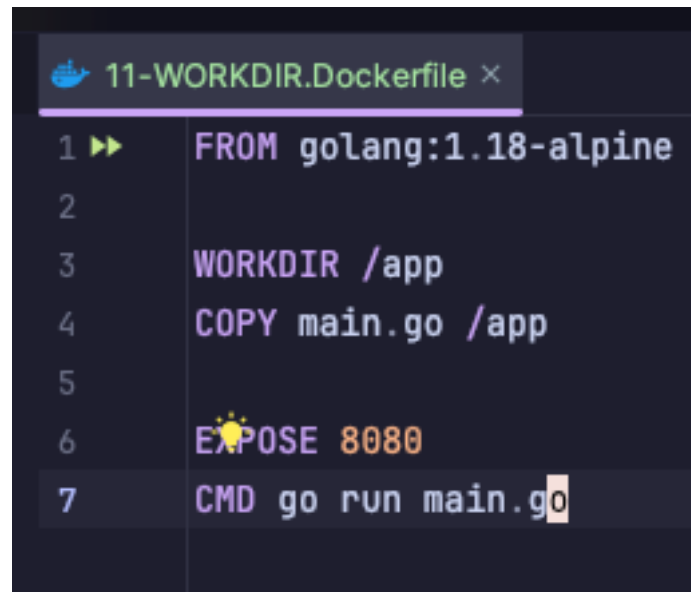
neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 20:01
> docker container logs volume
Run app in port : 8080
DONE Write File : /logs/chill.txt
DONE Write File : /logs/favicon.ico.txt

neru .../parallel-and-distributed-systems/hands-on-2 P main x!+? 20:02
> docker container inspect volume | grep -A 10 "Mounts"
"Mounts": [
  {
    "Type": "volume",
    "Name": "8a777de3ebc9d5870d47e10dc0a05b277268c04d5175205bb6017fcbca6cf7bfb",
    "Source": "/var/lib/docker/volumes/8a777de3ebc9d5870d47e10dc0a05b277268c04d5175205bb6017fcbca6cf7bfb/_data",
    "Destination": "/logs",
    "Driver": "local",
    "Mode": "",
    "RW": true,
    "Propagation": ""
  }
]

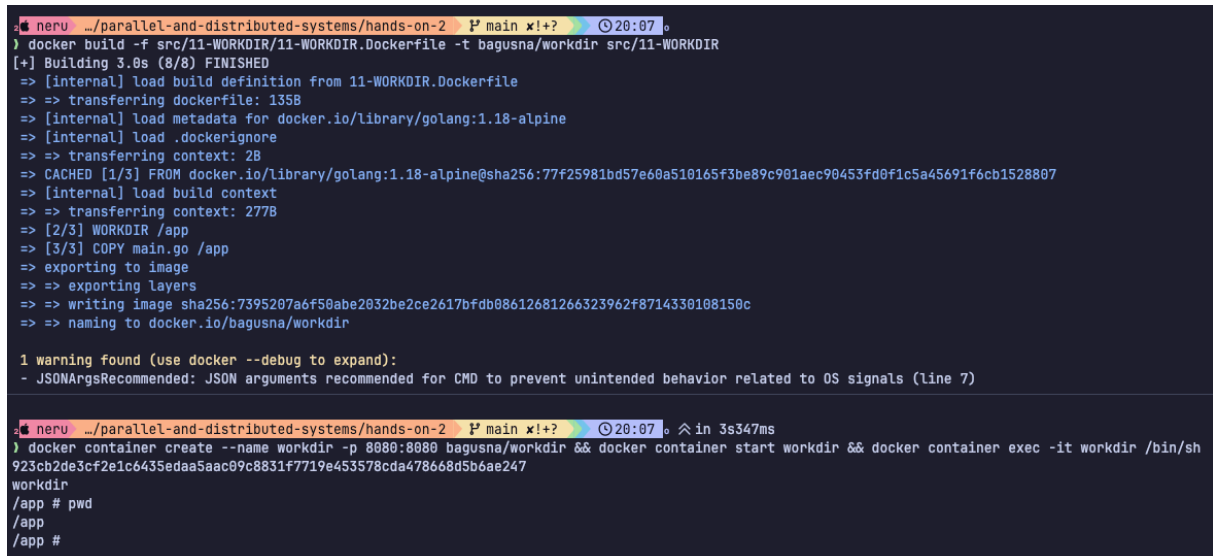
```

Instruksi VOLUME digunakan untuk membuat volume secara otomatis ketika container dibuat. Dalam contoh ini, digunakan volume `/logs`.

WORKDIR Instruction



```
11-WORKDIR.Dockerfile x
1  FROM golang:1.18-alpine
2
3  WORKDIR /app
4  COPY main.go /app
5
6  EXPOSE 8080
7  CMD go run main.go
```



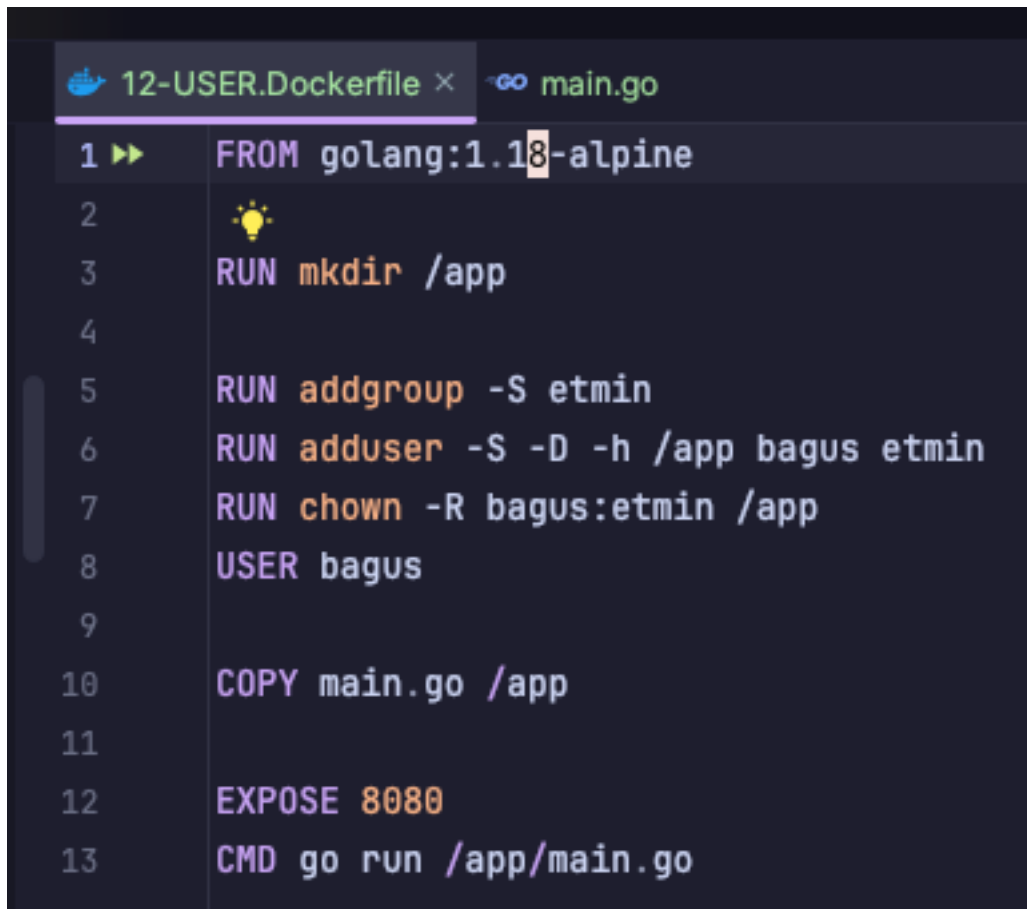
```
neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:07
$ docker build -f src/11-WORKDIR/11-WORKDIR.Dockerfile -t bagusna/workdir src/11-WORKDIR
[+] Building 3.8s (8/8) FINISHED
=> [internal] load build definition from 11-WORKDIR.Dockerfile
=> => transferring dockerfile: 135B
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/3] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> [internal] load build context
=> => transferring context: 277B
=> [2/3] WORKDIR /app
=> [3/3] COPY main.go /app
=> exporting to image
=> => exporting layers
=> => writing image sha256:7395207a6f50abe2032be2ce2617b9fdb08612681266323962f8714330108150c
=> => naming to docker.io/bagusna/workdir

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 7)

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:07 in 3s347ms
$ docker container create --name workdir -p 8080:8080 bagusna/workdir && docker container start workdir && docker container exec -it workdir /bin/sh
workdir
/app # pwd
/app
/app #
```

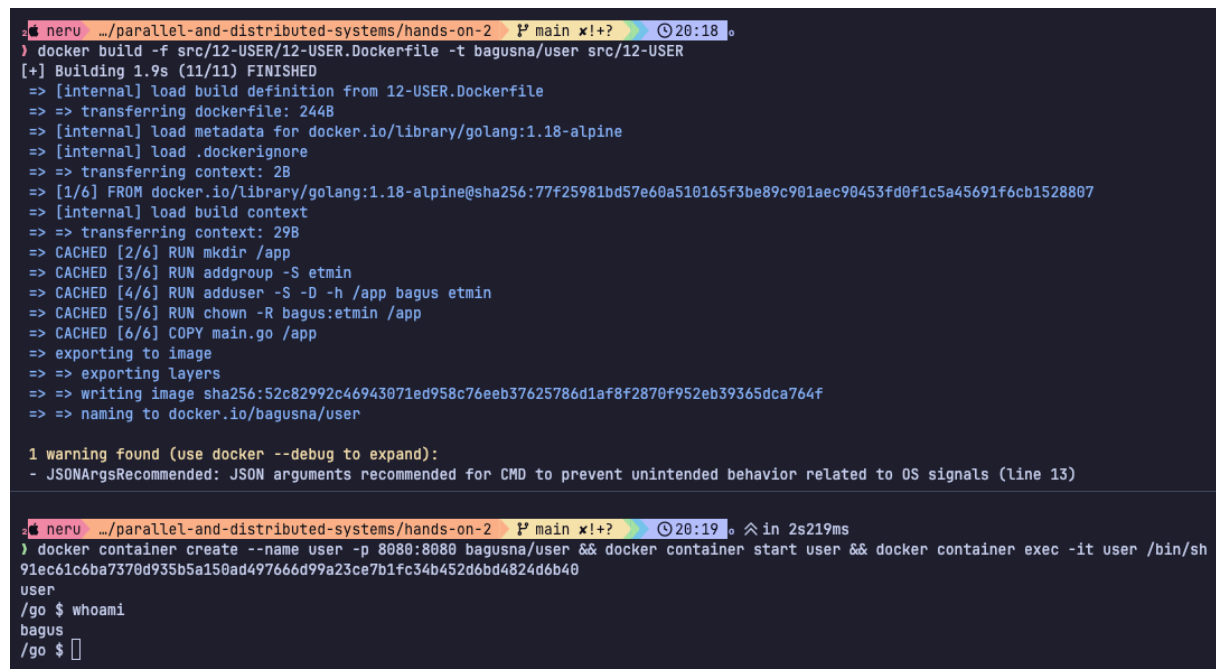
Instruksi WORKDIR berfungsi untuk menentukan directory berjalannya instruksi RUN, CMD, ENTRYPOINT, COPY, dan ADD.

USER Instruction



```

1  FROM golang:1.18-alpine
2
3  RUN mkdir /app
4
5  RUN addgroup -S etmin
6  RUN adduser -S -D -h /app bagus etmin
7  RUN chown -R bagus:etmin /app
8  USER bagus
9
10 COPY main.go /app
11
12 EXPOSE 8080
13 CMD go run /app/main.go
  
```



```

$ docker build -f src/12-USER/12-USER.Dockerfile -t bagusna/user src/12-USER
[+] Building 1.9s (11/11) FINISHED
=> [internal] load build definition from 12-USER.Dockerfile
=> => transferring dockerfile: 244B
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> [internal] load build context
=> => transferring context: 29B
=> CACHED [2/6] RUN mkdir /app
=> CACHED [3/6] RUN addgroup -S etmin
=> CACHED [4/6] RUN adduser -S -D -h /app bagus etmin
=> CACHED [5/6] RUN chown -R bagus:etmin /app
=> CACHED [6/6] COPY main.go /app
=> exporting to image
=> => exporting layers
=> => writing image sha256:52c82992c46943071ed958c76eeb37625786d1af8f2870f952eb39365dca764f
=> => naming to docker.io/bagusna/user

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 13)

$ docker container create --name user -p 8080:8080 bagusna/user && docker container start user && docker container exec -it user /bin/sh
user
/go $ whoami
bagus
/go $
  
```

Instruksi USER berfungsi untuk mengubah user maupun user group untuk image yang dibangun. Secara default, docker menggunakan `root` user dan dapat memberikan beberapa *concern* untuk use case tertentu. Dalam contoh, digunakan user `bagus` dengan user group `etmin` sebagai pengganti.

ARG Instruction

```

13-ARG.Dockerfile x
1 FROM golang:1.18-alpine
2
3 ARG app=main
4
5 RUN mkdir app
6 COPY main.go app
7 RUN mv app/main.go app/${app}.go
8
9 EXPOSE 8080
10
11 ENV app=${app}
12 CMD go run app/${app}.go

```

```

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:24
> docker build -f src/13-ARG/13-ARG.Dockerfile -t bagusna/arg src/13-ARG --build-arg app=wow
[+] Building 2.5s (9/9) FINISHED
=> [internal] load build definition from 13-ARG.Dockerfile
=> => transferring dockerfile: 202B
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/4] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> [internal] load build context
=> => transferring context: 29B
=> [2/4] RUN mkdir app
=> [3/4] COPY main.go app
=> [4/4] RUN mv app/main.go app/wow.go
=> exporting to image
=> => exporting layers
=> => writing image sha256:578b5c0193b266ebe8469fc4ab0a7e38954457c2bb63c937a5629d5d83a2ec63
=> => naming to docker.io/bagusna/arg

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 12)

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:25 in 2s800ms
> docker container create --name arg -p 8080:8080 bagusna/arg && docker container start arg && docker container exec -it arg /bin/sh
arg
/go # ls
app bin src
/go # ls app/
wow.go
/go #

```

ARG digunakan untuk mendefinisikan variabel yang dapat ditimpa oleh pengguna dalam proses build. Mirip dengan ENV, ARG sangat berguna untuk mengirimkan parameter atau data yang diperlukan dalam proses build image yang tidak dapat di-*hardcode* langsung. Lifetime ARG hanya selama proses build image dan tidak sampai proses run container.

HEALTHCHECK Instruction

```

13-ARG.Dockerfile 14-HEALTHCHECK.Dockerfile x main.go
1 FROM golang:1.18-alpine
2
3 RUN apk --no-cache add curl
4 RUN mkdir app
5 COPY main.go app
6
7 EXPOSE 8000
8
9 HEALTHCHECK --interval=5s --start-period=5s CMD curl -f http://localhost:8000/health
10
11 CMD go run app/main.go
12

```

```

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:39
$ docker build -f src/14-HEALTHCHECK/14-HEALTHCHECK.Dockerfile -t bagusna/health src/14-HEALTHCHECK
[+] Building 2.2s (9/9) FINISHED
=> [internal] load build definition from 14-HEALTHCHECK.Dockerfile
=> => transferring dockerfile: 2618
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 28
=> [1/4] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901a9c98453fd0f1c5a45691f6cb1528807
=> [internal] load build context
=> => transferring context: 298
=> CACHED [2/4] RUN apk --no-cache add curl
=> CACHED [3/4] RUN mkdir app
=> CACHED [4/4] COPY main.go app
=> exporting to image
=> => exporting layers
=> => writing image sha256:4a6d825e6f93237fc62c76907af77ba78d38d258c18eac16811c69114e57f389
=> => naming to docker.io/bagusna/health

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 11)

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:39
$ docker container create --name health -p 8080:8080 bagusna/health && docker container start health && sleep 2 && docker container ls && sleep 6 && docker container ls
health
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
5856263cc247   bagusna/health   "/bin/sh -c 'go run -"   Less than a second ago   Up Less than a second (health: starting)   8080/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   health
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
5856263cc247   bagusna/health   "/bin/sh -c 'go run -"   2 seconds ago   Up 2 seconds (health: starting)   8080/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   health
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
5856263cc247   bagusna/health   "/bin/sh -c 'go run -"   8 seconds ago   Up 8 seconds (healthy)   8080/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   health

```

```

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:39
$ docker container inspect health | grep -A 11 "Health"
    "Health": {
      "Status": "Unhealthy",
      "FailingStreak": 16,
      "Log": [
        {
          "Start": "2026-05-19T20:40:49.246836396+08:00",
          "End": "2026-05-19T20:40:49.283949733+08:00",
          "ExitCode": 22,
          "Output": " % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current\n 0      0  ---:--:  ---:--:  ---:--:  0\r\n 0      0  0     0  0     0  0     0  0     0  0     0  0     0  0     0  0\n",
          "Start": "2026-05-19T20:40:54.289353117+08:00",
        }
      ],
      "Healthcheck": {
        "Test": [
          "CMD-SHELL",
          "curl -f http://localhost:8080/health"
        ],
        "Interval": "5000000000",
        "StartPeriod": "5000000000"
      },
      "Image": "bagusna/health",
      "Volumes": null,
      "WorkingDir": "/go",
      "Entrypoint": null,
    },
  ],
}

```

HEALTHCHECK digunakan untuk mengecek status *ready* (*healthy*) atau tidaknya (*unhealthy*) container. Umumnya digunakan endpoint khusus yang akan dipanggil melalui HEALTHCHECK secara berkala.

ENTRYPOINT Instruction

```
15-ENTRYPOINT.Dockerfile × main.go
1 FROM golang:1.18-alpine
2
3 RUN mkdir /app/
4 COPY main.go /app/
5
6 EXPOSE 8080
7 ENTRYPOINT ["go", "run"]
8 CMD ["/app/main.go"]
```

```
neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:42
} docker build -f src/15-ENTRYPOINT/15-ENTRYPOINT.Dockerfile -t bagusna/entrypoint src/15-ENTRYPOINT
[+] Building 2.8s (8/8) FINISHED
=> [internal] load build definition from 15-ENTRYPOINT.Dockerfile
=> => transferring dockerfile: 169B
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 28
=> CACHED [1/3] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e68a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> [internal] load build context
=> => transferring context: 277B
=> [2/3] RUN mkdir /app/
=> [3/3] COPY main.go /app/
=> exporting to image
=> => exporting layers
=> writing image sha256:449892b31835a50d87df83784bc56451c828436c23bec638f446c7525779326f
=> naming to docker.io/bagusna/entrypoint

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:43 in 3s154ms
} docker container create --name entrypoint -p 8080:8080 bagusna/entrypoint && docker container start entrypoint && docker container exec -it entrypoint /bin/sh
entrypoint
/go # exit
```

ENTRYPOINT berfungsi untuk menentukan executable file yang akan dijalankan ketika container berjalan. Pada contoh ini, `go run` akan berjalan ketika container dimulai.

Multi-Stage Dockerfile

```

16-multistage.Dockerfile x
1  FROM golang:1.18-alpine AS builder
2  WORKDIR /app/
3  COPY main.go .
4  RUN go build -o /app/main main.go
5
6  FROM alpine:3
7  WORKDIR /app/
8  COPY --from=builder /app/main .
9  CMD /app/main

```

```

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:49
) docker build -f src/16-multistage.Dockerfile -t bagusna/multistage src/16-multistage
[+] Building 1.0s (13/13) FINISHED
=> [internal] load build definition from 16-multistage.Dockerfile
=> => transferring dockerfile: 226B
=> [internal] load metadata for docker.io/library/alpine:3
=> [internal] load metadata for docker.io/library/golang:1.18-alpine
=> [internal] load .dockerignore
=> => transferring context: 28
=> [builder 1/4] FROM docker.io/library/golang:1.18-alpine@sha256:77f25981bd57e60a510165f3be89c901aec90453fd0f1c5a45691f6cb1528807
=> [stage-1 1/3] FROM docker.io/library/alpine:3@sha256:5b18f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
=> [internal] load build context
=> => transferring context: 29B
=> CACHED [stage-1 2/3] WORKDIR /app/
=> CACHED [builder 2/4] WORKDIR /app/
=> CACHED [builder 3/4] COPY main.go .
=> CACHED [builder 4/4] RUN go build -o /app/main main.go
=> CACHED [stage-1 3/3] COPY --from=builder /app/main .
=> exporting to image
=> exporting layers
=> => writing image sha256:0f2cb8e13a967734fa089cdd145f9d5de7f9dbf8c85bb29bc61edc6078b32b0d
=> => naming to docker.io/bagusna/multistage

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 9)

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:49
) docker image ls | grep multistage
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
bagusna/multistage:latest          0f2cb8e13a96          14.7MB          0B          U

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:49
) docker container create --name multi -p 8080:8080 bagusna/multistage && docker container start multi && docker container exec -it multi /bin/sh
Error response from daemon: Conflict. The container name "/multi" is already in use by container "52375c7e8ac184834ef1ecc7b681eafabf68a593dfcd9058hat name".

neru ~/parallel-and-distributed-systems/hands-on-2 P main x!+? 20:49
) docker container create --name multi -p 8080:8080 bagusna/multistage && docker container start multi && docker container exec -it multi /bin/sh
multi
/app # ps aux
PID USER TIME COMMAND
1 root 0:00 /app/main
12 root 0:00 /bin/sh
18 root 0:00 ps aux
/app #

```

Untuk meminimalkan besaran image yang dihasilkan, dapat dilakukan approach multi-stage. Tiap stage dalam Dockerfile melakukan pengolahan menggunakan base image yang sesuai, dengan stage terakhir akan digunakan

sebagai base image dalam image yang dibuat. Misalnya, digunakan dua stage terpisah project frontend berbasis Vite. Stage pertama digunakan untuk mem-*build* project dengan base image dari Node.js. Sedangkan stage kedua cukup meng-*copy* build file dari stage sebelumnya dan menyajikannya melalui web server dengan base image Nginx atau Caddy.